IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 24 November 2025, accepted 19 December 2025, date of publication 25 December 2025,
date of current version 21 January 2026.

Digital Object Identifier 10.1109/ACCESS.2025.3648573

RESEARCH ARTICLE

Explainability in Software Architectural Decisions: The ADR-E Framework and Empirical Evaluation

RICARDO GACITÚA[1], JAVIER PEREIRA[2], AND MICHAEL KLAFFT[3,4]
[1]Department of Computer Science and Informatics, Universidad de La Frontera, Temuco 4811230, Chile
[2]ITiSB, Universidad Andrés Bello, Santiago 2520000, Chile
[3]Jade University of Applied Sciences, 26389 Wilhelmshaven, Germany
[4]Fraunhofer FOKUS, Berlin, Germany

Corresponding author: Ricardo Gacitúa (ricardo.gacitua@ufrontera.cl)

ABSTRACT As software systems increase in scale and complexity, architectural decisions must be transparent, traceable, and understandable to diverse stakeholders. However, traditional documentation approaches—such as standard Architectural Decision Records (ADRs)—often lack the structured rationale and contextual detail necessary to support informed analysis and long-term architectural stewardship. This paper presents the **Software Architecture Explainability Framework (SAEF)**, a structured approach for enabling explainable architectural decision-making. Central to the framework is the **Explainable Architectural Decision Record (ADR-E)**, which extends traditional ADRs with explicit rationale, structured stakeholder-oriented explanations, rejected alternatives, and traceability links grounded in explainability principles inspired by AI. SAEF was evaluated through two industrial case studies: the selection of Azure Kubernetes Service for container orchestration and the adoption of an enterprise-grade observability platform. Using a mixed-methods design combining workshops, scenario-based simulations, surveys, interviews, and operational metrics, the study found that ADR-E substantially improved transparency, traceability, and stakeholder alignment. Both cases reported a 30% reduction in mean time to resolution (MTTR) and transparency scores above 4.6/5. Overall, SAEF provides a practical and theoretically grounded foundation for explainable architectural decision-making. Future work will focus on tool support, graphical notations, and longitudinal assessments to enhance adoption and scalability.

INDEX TERMS Software architecture explainability transparency artificial intelligence.

I. INTRODUCTION

Modern software systems are characterised by increasing complexity, diverse stakeholder involvement, and continuous architectural evolution. In this context, architectural decisions play a central role in shaping system qualities, yet their rationale often remains implicit or insufficiently documented [1], [2]. While transparency in decision-making is necessary, transparency alone—the mere availability of information—is insufficient. What practitioners increasingly require is *explainability*: the ability to make architectural decisions understandable, justified, and traceable.

Explainability is a well-established concept in Artificial Intelligence (AI), where it supports understanding of why a model behaves as it does [3], [4]. Techniques such as feature attribution and model-agnostic explanations demonstrate how clarity and justification can increase trust and support responsible decision-making. Software architecture faces a comparable challenge: articulating why decisions are made, which alternatives were considered, and how these decisions influence system qualities. Although concepts such as transparency, interpretability, traceability, and observability are frequently discussed in the architecture literature [5], their relationship with explainability remains insufficiently explored.

Despite decades of progress in architectural documentation, traditional approaches—including mainstream Architectural Decision Records (ADRs)—often fail to capture complete rationales, dependencies, or context-sensitive explanations. Architectural knowledge is frequently

The associate editor coordinating the review of this manuscript and approving it for publication was Fabrizio Messina.

scattered, incomplete, or implicit, affecting comprehension and collaboration [6], [7]. As a consequence, stakeholders often struggle to understand architectural choices, particularly in large-scale or rapidly evolving systems.

This lack of explainability has tangible practical consequences, including misalignment between technical and business stakeholders, reduced traceability and limited impact analysis, difficulties in onboarding new team members, inconsistencies during maintenance and evolution, and increased risk of non-compliance in regulated domains. These issues are consistent with empirical studies showing that poorly justified architectural decisions contribute to architectural drift, rework, and communication breakdowns [8], [9]. Consequently, a structured and stakeholder-oriented approach to architectural explainability is increasingly needed.

Explainability in software architecture refers to the extent to which architectural decisions, their rationale, and their anticipated impact can be understood by diverse stakeholders. An explainable architecture communicates not only *what* decisions were taken, but also *why*, *how*, and *under what assumptions*. This clarity supports stakeholder alignment, traceability, and impact analysis, long-term maintainability, and accountability in regulated environments.

Existing work on ADRs, architectural knowledge sharing, and decision modelling provides valuable foundations; however, important limitations persist. Traditional ADR templates primarily focus on documenting decision outcomes, while often under-specifying the rationale and assumptions behind decisions, links to related decisions and architectural concerns, the questions stakeholders may ask (e.g., *Why this? What if? Who is affected?*), and the purposes for explaining a decision (e.g., justification, evaluation, or learning). Moreover, although explainability frameworks are prominent in AI, there is no established method that systematically adapts these principles to architectural decision documentation.

To address these limitations, this paper proposes the **Software Architecture Explainability Framework (SAEF)**, which adapts the core principles of AI explainability to software architecture. At the centre of SAEF is the *Architectural Decision Record – Explainable (ADR-E)*, an enhanced decision template that integrates structured rationales, explicit links to related architectural decisions, stakeholder-oriented explanations, decision-oriented research questions (*Why, What, What-if, Who, Where, When*), and explicit purposes for explaining decisions. The framework is validated through two empirical case studies in industrial settings, examining how ADR-E supports transparency, traceability, and stakeholder understanding.

By grounded architectural decisions in structured explainability principles, this work contributes to improving communication, accountability, and long-term architectural sustainability. SAEF is intended to complement existing documentation practices and support more transparent, explainable, and evidence-driven architectural decision-making. To facilitate practical adoption, a set of practitioner-oriented guidelines is provided in Appendix A.

The remainder of this paper is organised as follows. Section II reviews related work on explainability and architectural decision documentation. Section III presents the Software Architecture Explainability Framework and the ADR-E template. Section IV describes the validation methodology. Section V reports findings from two industrial case studies. Section VI discusses implications, limitations, and lessons learnt. Section VIII outlines future work, and Section IX concludes the paper.

II. RELATED WORK

Explainability intersects two major bodies of knowledge: (1) architectural knowledge and decision documentation in software architecture, and (2) explainability methods originating in Artificial Intelligence (AI). This section reviews these foundations, highlights limitations in existing approaches, and positions the proposed framework within this landscape.

A. ARCHITECTURAL KNOWLEDGE AND DECISION DOCUMENTATION

Architectural decisions have long been recognized as first-class entities in software architecture. The Architectural Knowledge (AK) community proposes several mechanisms to document, communicate, and reason about such decisions.

Early work by Jansen et al. [10] used viewpoints and perspectives to make architectural concerns explicit to different stakeholders. While valuable for structuring views, this approach does not provide detailed decision rationales or guidance on how to explain trade-offs.

Tyree and Akerman's widely adopted Architectural Decision Records (ADRs) [6] introduced a lightweight, structured way to document decisions, including context, alternatives, and consequences. Over time, variations such as MADR (Markdown Architectural Decision Records) emerged, offering standardized formats and tool support (e.g., ADR Manager). Although these templates improved consistency, they primarily capture *what* was decided, with limited emphasis on *why* or *how* to communicate decisions to diverse stakeholders.

Zimmermann et al. [7] further contributed decision patterns and guidance for cross-project knowledge reuse. Their work emphasizes pattern-based documentation but does not explicitly address explainability, stakeholder-oriented explanation, or structured rationale.

Beyond ADRs, several lightweight approaches, such as Y-statements, aim to reduce documentation overhead and capture decisions succinctly. While effective for agile teams, these approaches may omit the depth of reasoning required in large-scale systems or regulated domains.

A consistent limitation across these efforts is the absence of a systematic method for ensuring that decisions are *explainable*—understandable, justified, contextualized, and

traceable across the architectural lifecycle. Existing templates provide structure, but not mechanisms to ensure clarity, sufficiency of explanation, or alignment with stakeholder needs.

B. EXPLAINABILITY-BY-DESIGN AND TRANSPARENCY IN ARCHITECTURE

The growing interest in transparency and interpretability has influenced recent architectural research. For instance, the Explainability-by-Design approach [11] integrates explainability considerations into architectural processes. Although promising, current proposals remain conceptual and lack standardised templates or operational guidance to document explanation-orientated decisions. Reviewers of such approaches also highlight their dependence on expert interpretation and the absence of tools or metrics to evaluate the quality of the explanation.

Similarly, work on observability and monitoring (e.g., [5]) focusses on operational transparency rather than decision transparency. These studies improve runtime understanding of system behaviour but do not address how or why architectural decisions should be explained.

Overall, although the importance of explainability is recognized, existing research does not provide a concrete, stakeholder-accessible mechanism for integrating explainability into architectural decision documentation.

C. EXPLAINABILITY IN ARTIFICIAL INTELLIGENCE

Explainability has a strong foundation in the AI community, where the need to make opaque models understandable has led to extensive research on explainable AI (XAI). Gilpin et al. [12] classify explainability methods into three major categories:

- **Transparent models:** inherently interpretable models such as decision trees or linear models.
- **Post-hoc explanations:** techniques such as LIME [13] and SHAP [14] that generate explanations without altering the model.
- **Example-based explanations:** methods that use representative examples or counterfactuals to illustrate model behaviour.

Arrieta et al. [3] emphasize that explanations must be comprehensible, actionable, and tailored to stakeholders, including non-technical users. They also highlight that explainability is closely linked to accountability, fairness, and trust.

Although these principles stem from AI, they provide relevant conceptual foundations for software architecture. In particular, stakeholder-centred communication, structured rationale, and traceability of decision-making are directly applicable to architectural documentation.

D. GAPS IN THE LITERATURE

Despite the breadth of research in AK documentation and XAI, several gaps persist.

- **Lack of structured explainability in architectural decisions.** Existing ADR templates and AK approaches document decisions but rarely guide how to explain them, how to justify them, or how to adapt explanations to different stakeholders.
- **Missing integration of explainability principles from AI.** Although AI explainability offers strong conceptual models (e.g., post-hoc explanations, example-based reasoning, stakeholder-oriented communication), these ideas have not been systematically adapted to architectural documentation.
- **Limited standardization.** Existing decision documentation practices vary widely across organizations. No established method defines how to embed explainability systematically into architectural decision processes.
- **Weak support for stakeholder comprehension.** Many ADR formats assume technical audiences and do not provide mechanisms to ensure that explanations are accessible, complete, or tailored to different viewpoints.
- **Lack of empirical validation.** Most explainability-by-design approaches remain conceptual and lack practical evaluations in real-world architectural settings.

E. COMPARISON OF EXISTING ADR APPROACHES FROM AN EXPLAINABILITY PERSPECTIVE

Architectural Decision Records (ADRs) are widely used to document and communicate architectural decisions, yet their ability to support *explainability*—that is, to make decisions understandable, justifiable and traceable for diverse stakeholders—varies substantially between approaches. To make these differences explicit, this subsection compares representative ADR approaches using an objective template-level protocol. The goal is not to assess decision quality or organisational maturity, but to identify whether existing approaches structurally support the documentation needs related to explainability.

1) COMPARISON PROTOCOL

The comparison focusses on *the explicit documentation capabilities* provided by each approach. We evaluate whether an approach includes dedicated fields, headings, or prompts intended to capture explainability-relevant information. The assessment is deterministic and does not rely on subjective judgement of how well a field is used in practice.

a: APPROACHES COMPARED

The following ADR approaches are included:

- ADR (T yree & Akerman) [6]: the original and widely adopted ADR template.
- MADR [15]: a standardised Markdown-based ADR variant emphasizing lightweight documentation.
- Y-Statements [7]: concise decision statements commonly used in agile environments.

- Explainability-by-design [11]: a representative explainability-by-design approach that promotes explainability as an architectural principle, but does not prescribe a concrete ADR template.

b: UNITS OF ANALYSIS

The unit of analysis is the *decision documentation structure* explicitly defined by each approach (i.e., the ADR template or prescribed decision representation), rather than associated tools, workflows, or empirical claims.

c: EVALUATION DIMENSIONS

Explainability-related support is operationalised using nine binary dimensions (D1–D9). Each dimension is coded as **1** if the approach provides an explicit structural element intended to capture that capability, and **0** otherwise.

To ensure objectivity and reproducibility, the following rules are applied:

- Explicitness rule: A capability is considered present only if the approach includes an explicit field, heading, or prompt intended to capture it.
- Non-inference rule: Capabilities that could be documented implicitly in free text but are not explicitly prompted are coded as absent.
- Template-level rule: For explainability-by-design approaches, only decision-record structures that are explicitly prescribed are evaluated; conceptual guidance without a concrete ADR template is coded as absent.

2) RESULTS OF THE COMPARISON

Table 1 summarises the results of applying the comparison protocol. The table shows that traditional ADR formats (ADR and MADR) consistently support key elements of the decision documentation, such as context, decision statements, alternatives, and consequences. However, they do not provide explicit structural support for explainability-oriented concerns, including stakeholder-specific explanations, documentation of rejected alternatives, explicit traceability across decisions and artefacts, or clarification of the purpose for explaining a decision.

Y-Statements intentionally minimise documentation structure to reduce overhead. Although effective in capturing the decision outcome, their single-sentence form does not provide explicit slots for rationale elaboration, alternative analysis, or stakeholder-orientated explanation, which limits their suitability in explainability-sensitive contexts.

Explainability-by-design approaches, exemplified by Brown and Lee [11], emphasise transparency and understandability as architectural principles. Nevertheless, these approaches remain largely conceptual and do not define a standardised decision-record structure. Consequently, explainability is not operationalised at the level of individual architectural decisions, leaving architects without concrete guidance on how to document explanations in a consistent and reviewable manner.

Overall, the comparison highlights a systematic limitation in existing ADR approaches: although they document *what* decision was made, they provide limited structural support to explain *why* it was made, *what alternatives were rejected*, *who is affected*, and *how the decision relates to other architectural concerns*. This limitation motivates the need for an explainability-orientated extension to the ADR documentation, which is addressed by the ADR-E template introduced in Section III.

F. POSITIONING OF THE PRESENT WORK

The proposed Software Architecture Explainability Framework (SAEF) addresses these gaps by introducing the *Architectural Decision Record – Explainable (ADR-E)*, which extends traditional ADRs with:

- structured, stakeholder-oriented explanations,
- explicit decision-oriented research questions (Why, What, What-if, Who, Where, When),
- clear purposes for explanation (justify, evaluate, manage, learn),
- enhanced rationale and traceability elements,
- and integration of explainability concepts inspired by AI.

Unlike existing documentation templates (ADR, MADR, Y-statements), ADR-E is explicitly designed to support decision explainability rather than decision recording alone. SAEF further provides a methodological context for applying ADR-E across the architectural lifecycle, grounding explainability in systematic, repeatable practices.

This combination of structured explanation, methodological integration, and stakeholder orientation differentiates the proposed approach from prior work and establishes a foundation for explainable architectural decision-making.

III. PROPOSAL: SOFTWARE ARCHITECTURE EXPLAINABILITY FRAMEWORK

This section presents the **Software Architecture Explainability Framework (SAEF)**, a structured approach for integrating explainability into architectural decision-making. SAEF builds on established practices from software architecture (e.g., ADRs, decision modelling, viewpoints) and adapts principles from explainable AI to support clearer, more accountable, and more stakeholder-oriented architectural decisions. The framework is designed to be lightweight, compatible with current industrial practice, and aligned with ISO 42010's emphasis on stakeholders, concerns, and architectural rationale.

A. FOUNDATION AND DESIGN PRINCIPLES

The derivation of SAEF follows three principles observed across the literature and practice:

1) **Architectural decisions require structured justification.** Empirical studies show that missing rationales and poorly documented assumptions hinder communication, impact analysis, and maintenance [6], [7].

TABLE 1. Comparison of existing ADR approaches using explainability-oriented documentation dimensions. "1" indicates that the capability is explicitly supported by the approach's decision documentation structure; "0" indicates that it is not explicitly supported. Explainability-by-design is evaluated with respect to the presence of a concrete, prescribed decision-record structure.

Explainability-related capability	ADR	MADR	Y-Statement	Explainability-by-design [11]
Core decision metadata (title, status, date)	1	1	0	0
Explicit decision statement	1	1	1	0
Context / problem description	1	1	0	0
Alternatives/options explicitly listed	1	1	0	0
Rejected alternatives with justification	0	0	0	0
Consequences and trade-offs	1	1	0	0
Traceability to related decisions and artefacts	0	0	0	0
Stakeholder-oriented explanation prompts	0	0	0	0
Explicit purpose for explaining a decision	0	0	0	0

2) **Explainability must be stakeholder-oriented.** Inspired by XAI, explanations must address the concerns of different stakeholders and adapt to their levels of technical expertise [3].
3) **Decision documentation must support learning and evolution.** Architecture is dynamic; explanations must be reviewable, comparable, and reusable across iterations and teams.

SAEF operationalizes these principles through a process model, a metamodel, and the enhanced ADR-E template that captures explainability elements missing in traditional ADR formats.

B. FRAMEWORK OVERVIEW

Figure 1 illustrates SAEF as an iterative and collaborative cycle embedded within the architectural workflow. The framework positions explainability as a continuous activity rather than a post-hoc documentation task.

SAEF consists of seven interconnected steps:

1) **Stakeholder Inputs:** Stakeholders articulate concerns, goals, expectations, constraints, questions and requirements. These elements define the explainability requirements associated with a decision.
2) **Architectural Design Process:** Consistent with ADD and ISO 42010, architects analyse concerns, explore alternatives, and evaluate trade-offs.
3) **Architectural Decisions:** Architectural choices are made based on rationale, quality attributes, constraints, and project goals.
4) **Explainability Process:** The architect develops structured explanations, records assumptions, articulates the rationale, and links the decision to related decisions and artefacts.
5) **Stakeholder Requests for Explanation:** Stakeholders may request tailored explanations addressing specific "Why?", "What-if?", and "Who is affected?" questions.
6) **Outputs for Development Phases:** Explainable decisions inform subsequent activities such as design, implementation, testing, and operationalization.
7) **Impact Assessment:** The consequences of the decision are reviewed and contrasted with original expectations, enabling learning and future refinement.

This cyclical structure supports decision awareness, traceability, and alignment throughout the lifecycle.

C. EXPLAINABILITY PROCESS

The **Explainability Process** is the core mechanism through which SAEF integrates explainability into architectural practices. It provides actionable steps to produce explanations that are complete, structured, and accessible.

The process comprises the following activities:

1) **Capture Explainability Requirements:** Stakeholders implicitly pose questions (e.g., *Why this approach? What alternatives exist? Who is impacted?*). SAEF makes these questions explicit and uses them to guide documentation.
2) **Generate and Evaluate Alternatives:** Architects analyse constraints and quality attributes, synthesise possible solutions, and evaluate trade-offs. SAEF requires documenting not only the chosen option but also rejected alternatives and the reasoning behind these choices.
3) **Document the Decision Using ADR-E:** The ADR-E template records rationale, assumptions, dependencies, and purposes for explanation, creating a traceable and comprehensible artefact.
4) **Produce Stakeholder-Oriented Explanations:** Explanations are framed using structured questions (Why, What-if, When, Who), enabling different stakeholders to understand the decision from their perspective.
5) **Review and Refine:** A feedback loop allows stakeholders to validate explanations, identify ambiguities, and request additional clarity. Iterative improvement strengthens alignment and shared understanding.

This process ensures that explainability is consistently maintained and not merely appended to documentation efforts late in the lifecycle.

D. METAMODEL FOR EXPLAINABILITY

The **Explainability Metamodel** (Figure 2) formalizes the relationships between stakeholders, architectural decisions, artefacts, and explainability elements. It is designed to complement ISO 42010 by extending the decision viewpoint with explainability constructs.

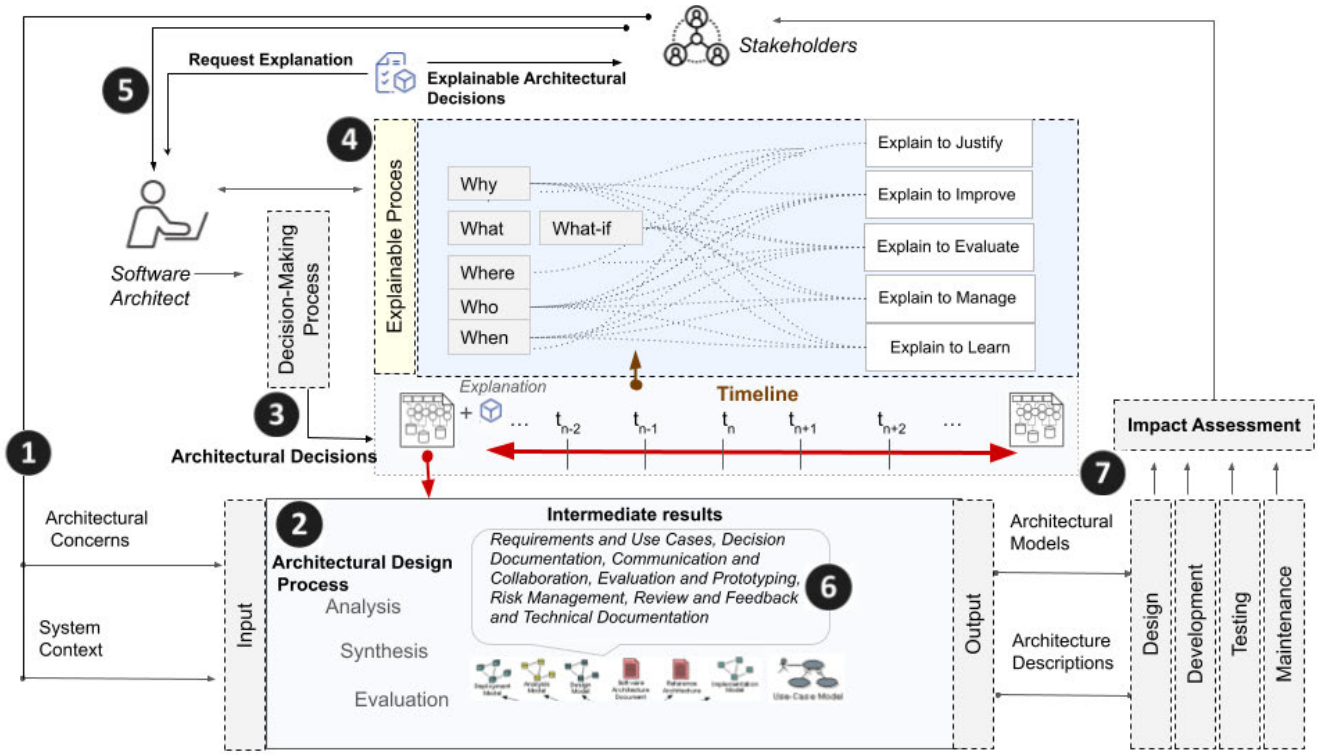

FIGURE 1. Conceptual overview of the software architecture explainability framework (SAEF).

Key relationships include:

- **Stakeholders** provide concerns and request explanations.
- **Architectural Decisions** include rationale, alternatives, consequences, and explainability attributes.
- **Artefacts** (models, code, requirements) contextualize decisions and serve as references in explanations.
- **Explainability Attributes** (traceability, comprehensibility, contextuality) capture quality aspects of the explanation itself.

The metamodel operationalizes explainability by linking decisions to concerns, artefacts, and explanatory content, enabling systematic navigation across them.

E. ENHANCED ARCHITECTURAL DECISION RECORD (ADR-E)

At the centre of SAEF is the **Enhanced Architectural Decision Record (ADR-E)**, a structured template that extends traditional ADRs with explainability-oriented elements derived from AK research and XAI principles.

Table 2 summarizes the key elements of ADR-E.

Whereas traditional ADRs focus primarily on documenting decisions, ADR-E explicitly supports understanding and communication. This shift from *recording* to *explaining* is at the heart of SAEF's contribution.

F. EXPECTED OUTCOMES AND BENEFITS

By embedding explainability into the architectural workflow, SAEF offers several benefits:

- **Improved Transparency:** Stakeholders gain a clearer understanding of why decisions were made.
- **Enhanced Traceability:** Relationships among concerns, rationale, artefacts, and decisions become explicit.
- **Stakeholder Alignment:** Explanations tailored to different audiences promote shared understanding.
- **Scalability and Reuse:** ADR-E artefacts can be compared, reviewed, and reused across projects, strengthening organizational learning.
- **Better Impact Analysis:** Structured rationale helps anticipate consequences and supports informed change decisions.

Together, these outcomes demonstrate the potential of SAEF to improve architectural communication, decision quality, and long-term system sustainability.

IV. METHODOLOGY

This section describes the methodological approach used to (1) design the Software Architecture Explainability Framework (SAEF) and the ADR-E template, and (2) evaluate their effectiveness in real software engineering contexts. The methodology adopts a hybrid research strategy that integrates a **Design Science** process for artefact construction with a

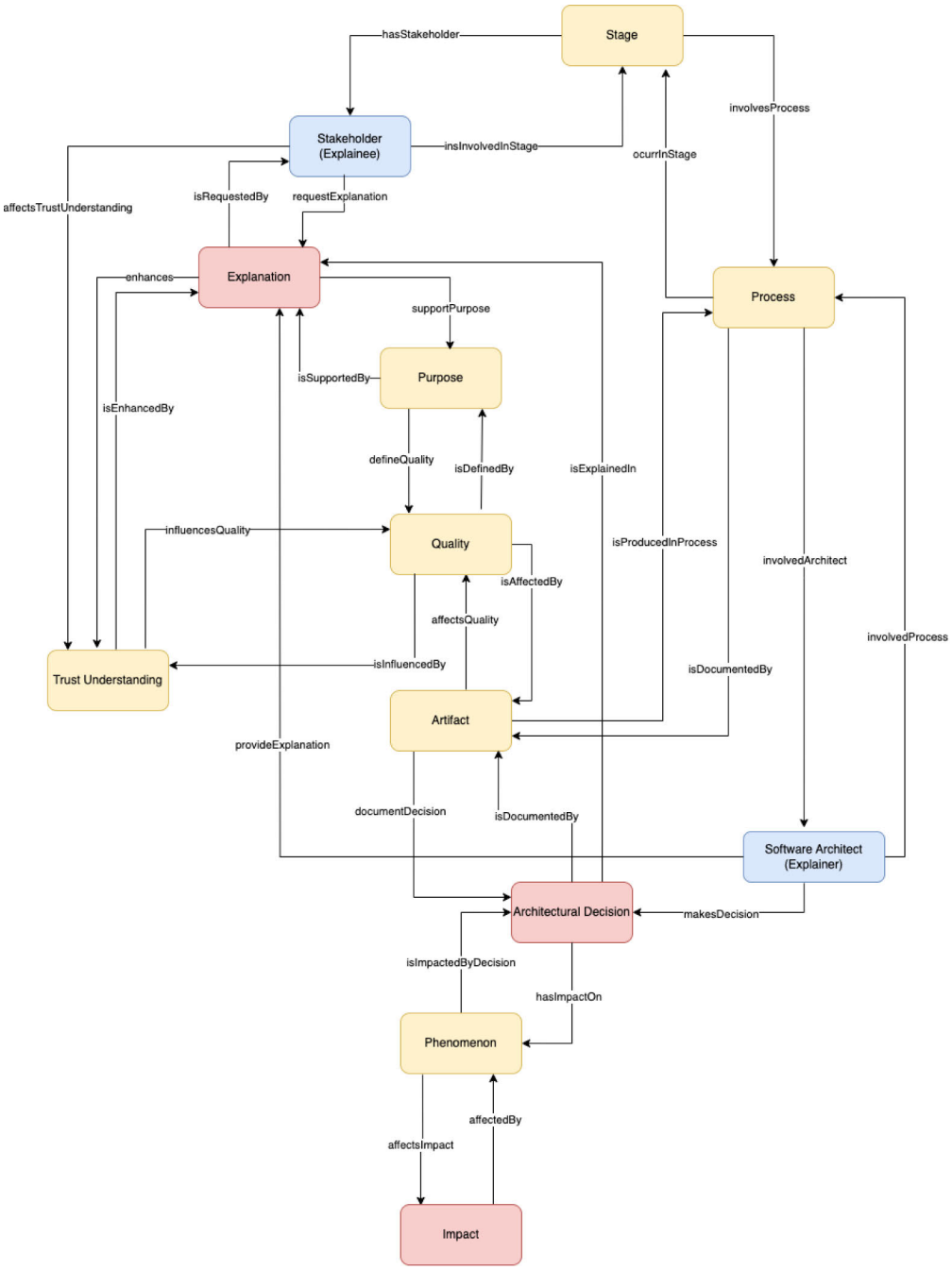

FIGURE 2. Metamodel capturing relationships among decisions, stakeholders, artefacts, and explainability elements.

TABLE 2. Explainability-oriented elements incorporated into the ADR-E template.

[HTML]EFEFEF **Element**	**Description**
Rationale	Comprehensive explanation of the reasoning, assumptions, and trade-offs.
Structured Explanation	Stakeholder-oriented explanation guided by questions such as Why, What-if, Who, Where, and When.
Purpose for Explaining	Clarifies whether the explanation aims to justify, evaluate, manage, or enable learning.
Related ADRs	Explicit traceability links to dependent or conflicting decisions.
Research/Decision Questions	Prompts that ensure coverage of decision dimensions and concerns.
Revision History	Tracks evolution to support accountability and long-term maintainability.

mixed-methods empirical evaluation. This combination ensures theoretical grounding, methodological rigor, and practical relevance.

A. METHODOLOGICAL APPROACH

The research process is structured into three coherent phases aligned with Design Science principles:

1) **Problem Identification and Requirements Derivation:** We analysed shortcomings in existing architectural decision documentation practices (ADR, MADR, Y-statements, ISO 42010) and synthesised explainability concepts from XAI. This produced a set of *explainability requirements* (ER1–ER5) that served as the foundation for the design of SAEF and ADR-E.
2) **Framework and Artefact Design:** SAEF was developed iteratively, incorporating a process model, metamodel, and the ADR-E template. Design choices were validated through expert review sessions involving architects and decision-analysis specialists.
3) **Empirical Evaluation Through Case Studies:** The framework and ADR-E were evaluated in two industrial case studies using mixed-methods: structured workshops, interviews, surveys, and quantitative operational metrics.

This structured approach creates a traceable connection between theory, design decisions, and empirical evidence, ensuring reproducibility and methodological transparency.

B. EXPLAINABILITY REQUIREMENTS

Explainability in software architecture depends on clear stakeholder understanding of why and how decisions are made. The five categories of explainability requirements derived in Phase 1 are:

- **ER1 – Rationale Requirements:** Explanation of motivations, assumptions, and trade-offs.
- **ER2 – Alternative Requirements:** Visibility into considered alternatives and justification for rejections.
- **ER3 – Contextual Requirements:** Clarification of who is affected, where the decision applies, and when it is valid.
- **ER4 – Traceability Requirements:** Links to related ADRs, requirements, risks, and quality attributes.
- **ER5 – Stakeholder-Oriented Communication:** Explanations adapted to diverse technical and non-technical audiences.

These requirements guided the construction of SAEF's process model, metamodel, and the ADR-E template.

To support reproducibility and methodological transparency, the instruments used for data collection—including the survey questionnaire, the semi-structured interview guide, and the workshop agendas—are provided in Appendix B.

C. SAEF PROCESS DEFINITION

Figure 3 presents the methodological workflow through which explainability is integrated into architectural activities. The workflow operationalises the explainability requirements through six steps that structure how architects reason about, document, and communicate decisions.

1) STEP 1: STAKEHOLDER IDENTIFICATION AND INPUT COLLECTION

Architects identify the relevant actors and concerns:

- **Stakeholder Mapping:** architects, developers, SREs, managers, and business leaders.
- **Concern Elicitation:** workshops, interviews, requirement reviews.
- **Explainability Requirement Capture:** mapping stakeholder questions to ER1–ER5.

2) STEP 2: DECISION CONTEXT ANALYSIS

The architectural scope and constraints of the decision are defined:

- **Scope Definition:** platform selection, deployment models, observability, data flows.
- **Quality Attribute Mapping:** performance, scalability, maintainability, resilience.
- **Trade-off Analysis:** comparison of alternatives using architecturally significant criteria.

3) STEP 3: EXPLAINABILITY PROCESS APPLICATION

The decision is transformed into a structured explanation:

1) **Recognize:** identify the decision requiring documentation.
2) **Understand:** analyse rationale, constraints, and dependencies.
3) **Explain:** produce the ADR-E entry addressing ER1–ER5.
4) **Interpret:** validate that explanations are actionable and comprehensible.

4) STEP 4: ARTEFACT DEVELOPMENT AND DOCUMENTATION

Architectural knowledge is consolidated:

- **ADR-E Authoring:** document rationale, alternatives, implications, and research questions.
- **Traceability Enhancement:** include links to requirements, risks, and related ADRs.
- **Version Control:** maintain a history of decision evolution.

5) STEP 5: VALIDATION AND COMMUNICATION

Stakeholder understanding is assessed and refined:

- **Presentation:** summaries, diagrams, and reviews present decisions to varied audiences.
- **Feedback Gathering:** surveys and interviews assess clarity and utility.

6) STEP 6: ITERATIVE REFINEMENT

Explainability improves as the system evolves:

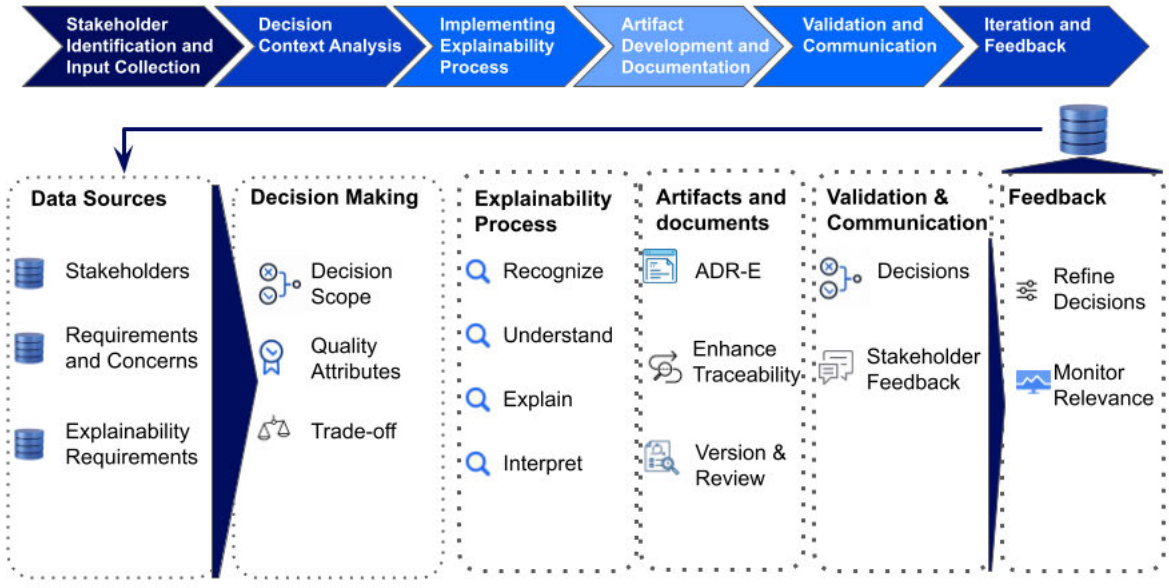

FIGURE 3. Methodological workflow for integrating explainability into software architecture.

- **Refinement:** update ADR-E entries and architectural artefacts based on feedback.
- **Reassessment:** confirm decision validity under evolving requirements or contexts.

D. EMPIRICAL EVALUATION DESIGN

Two industrial case studies were conducted to evaluate SAEF:

- **Quantitative Measures:** MTTR reduction, transparency and traceability scores, and comprehension ratings.
- **Qualitative Measures:** Semi-structured interviews, thematic coding, and workshop observations.
- **Triangulation:** Integration of metrics, perceptions, and artefact analysis to validate conclusions.

This mixed-methods design follows established empirical software engineering guidelines and ensures robustness and credibility in the evaluation.

E. EXPECTED OUTCOMES

The methodology supports:

- **Transparent Decisions:** structured explanations aligned with stakeholder concerns.
- **Traceable Knowledge:** explicit links connecting decisions, rationale, and quality attributes.
- **Stakeholder Trust:** improved confidence in architectural reasoning.
- **Higher-Quality Decisions:** more balanced evaluation of alternatives and impacts.
- **Sustained Architectural Knowledge:** reusable artefacts that support long-term maintainability.

The methodology provides a rigorous foundation for systematically integrating explainability into architectural decision-making. Providing these instruments makes the empirical evaluation replicable and allows other researchers to adapt the methodology to different architectural contexts.

V. CASE STUDIES: VALIDATING THE EXPLAINABILITY FRAMEWORK

This section validates the Software Architecture Explainability Framework (SAEF) through case studies conducted in two distinct scenarios. The validation process aimed to assess SAEF's practical utility in enhancing transparency, traceability, and stakeholder understanding.

A. CASE STUDY 1: SELECTING AZURE KUBERNETES SERVICE (AKS) FOR CONTAINER ORCHESTRATION

This case study evaluates the applicability and usefulness of the Software Architecture Explainability Framework (SAEF) and the ADR-E template in the context of a real architectural decision: selecting an orchestration platform for Kubernetes-based microservices. The study was conducted in a mid-sized enterprise undergoing a migration toward a cloud-native architecture.

The objective of Case Study 1 was to investigate whether ADR-E improves (1) stakeholder understanding of the rationale and trade-offs behind the decision, (2) traceability between architectural concerns and decision outcomes, and (3) the ability to assess the decision's operational impact.

1) CONTEXT AND DECISION PROBLEM

The organization planned to deploy a microservices architecture requiring automated scaling, resilience, and manageable operations. The architectural team needed to choose between:

- adopting **Azure Kubernetes Service (AKS)** (managed solution), or

- deploying **self-hosted Kubernetes** clusters (unmanaged solution).

This decision had significant implications for operational overhead, maintainability, vendor lock-in, and cost. Previous decisions in the organization had been documented using traditional ADRs, but these often lacked detailed rationale, explicit alternatives, and stakeholder-oriented explanations. As a result, misunderstandings arose and the reuse of architectural knowledge was limited.

Case Study 1 therefore served as an opportunity to apply ADR-E to a concrete architectural choice and to compare its outcomes with earlier ADR-based documentation practices.

The Table 3 illustrates the ADR-E for the architectural decision *Use AKS for Kubernetes Pods*. By incorporating these items into the ADR-E template, the decision is made transparent, traceable, and understandable. Each component contributes to a comprehensive documentation process that enhances stakeholder communication, supports strategic alignment, and ensures that decisions are well-founded and effectively communicated. This approach fosters a culture of transparency and continuous improvement in software architecture decision-making.

2) PARTICIPANTS

Six professionals participated in the study, representing heterogeneous roles to ensure diverse concerns:

- 1 Software Architect (decision owner),
- 2 Platform Engineers,
- 2 Site Reliability Engineers (SREs),
- 1 Customer Support Manager.

This distribution enabled the evaluation of ADR-E from both technical and non-technical perspectives, addressing a key explainability requirement emphasized in SAEF.

3) STUDY DESIGN AND PROCEDURE

The case study followed a structured sequence of activities aligned with SAEF and the mixed-methods research design.

a: 1) EXPLAINABILITY REQUIREMENTS ELICITATION

A facilitated workshop was used to identify stakeholder concerns regarding:

- operational burden,
- scalability under traffic surges,
- fault recovery behaviour,
- integration with existing Azure services,
- long-term vendor lock-in risks.

These concerns were mapped to ER1–ER5, providing the explainability requirements guiding the ADR-E construction.

b: 2) DECISION MODELLING AND ADR-E CONSTRUCTION

The architectural team analysed alternatives, defined evaluation criteria, and produced a complete ADR-E entry documenting:

- rationale and underlying assumptions,
- alternatives considered and rejected,
- consequences and expected system impact,
- structured explanations addressing *Why*, *What-if*, *Who*, *Where*, and *When*,
- explicit links to related decisions (e.g., cloud vendor selection).

For comparison, the same decision was also documented using a traditional ADR template, enabling baseline evaluation.

c: 3) VALIDATION WORKSHOPS

Three decision-review workshops were conducted:

- Workshop 1: presentation of context, alternatives, and trade-offs.
- Workshop 2: scenario-based simulation (e.g., scaling spikes, node failures).
- Workshop 3: stakeholder clarification and feedback round.

Participants used both ADR and ADR-E documentation to evaluate which representation better supported comprehension and traceability.

d: 4) QUANTITATIVE ASSESSMENT

We measured three metrics:

- **Transparency Score (TS):** Mean stakeholder rating (1–5 Likert) of explanation clarity.
- **Traceability Alignment (TA):** Independent reviewers scored the presence of links to concerns, requirements, quality attributes, and related decisions (scale 0–5).
- **Mean Time to Resolution (MTTR):** Fault injection experiments simulated pod failures before and after implementing AKS. MTTR is calculated as:

$$MTTR = \frac{\sum(\text{time to detect} + \text{time to recover})}{n}$$

e: 5) QUALITATIVE ASSESSMENT

Semi-structured interviews (20–30 minutes each) explored:

- the usefulness of explanations,
- clarity of trade-offs,
- sufficiency and trustworthiness of rationale,
- perceived improvements over traditional ADRs.

Responses were coded using thematic analysis.

4) RESULTS

Table 4 summarizes the quantitative outcomes.

a: TRANSPARENCY

Stakeholders reported that ADR-E's structured questions and explicit explanation sections significantly improved comprehension compared to the traditional ADR.

b: TRACEABILITY

ADR-E produced clearer links to quality attributes, requirements, and related decisions, addressing weaknesses frequently noted in previous documentation.

TABLE 3. ADR-E instance for the architectural decision: Use AKS for Kubernetes Pods (Case Study 1).

Element	Description
Title	ADR001 - Use AKS for Kubernetes Pods
Status	Adopted
Decision	Use Azure Kubernetes Service (AKS) for managing Kubernetes pods.
Context	The organization is transitioning to a microservices architecture and requires a scalable, manageable solution for orchestrating its containerized applications.
Options Considered	1. Use AKS (Adopted): Pros - Managed service, easy integration with Azure. Cons - Vendor lock-in. 2. Use self-hosted Kubernetes: Pros - Full control, no vendor lock-in. Cons - Higher maintenance overhead.
Consequences	Positive - Simplified management, scalability. Negative - Dependency on Azure infrastructure.
Advice	Advice given by Cloud Architect on 2024-06-17, recommending AKS for its integration capabilities and managed service benefits.
Rationale	AKS was chosen due to its managed service offering, reducing the maintenance burden and providing seamless integration with existing Azure resources.
Version	v1.0
Date Created	2024-06-17
Date Updated	2024-06-17
Related ADRs	ADR002 - Container Orchestration Strategy ADR003 - Cloud Vendor Selection
Explanation	AKS provides a robust, managed Kubernetes service, ideal for our microservices architecture due to its scalability and integration with Azure DevOps pipelines. This decision aligns with our strategic move towards cloud-native infrastructure.
Revision History	v1.0 (2024-06-17) - Initial creation.
Feedback and Review	Reviewed by DevOps Team Lead on 2024-06-18: Agreement on the decision.
Research Questions	**Why:** Need for scalable, managed Kubernetes services. **What:** Use AKS for managing Kubernetes pods. **What-if:** If we used self-hosted Kubernetes, it would increase maintenance overhead. **Where:** Applies to the container orchestration layer. **Who:** Software Architects, DevOps Team. **When:** Decision made during the initial architecture phase, 2024-06-17.
Purpose for Explaining	**Justify:** Provides reasons for choosing AKS over alternatives. **Improve:** Helps enhance future decision-making processes. **Evaluate:** Assesses the decision's impact on system scalability and maintenance. **Manage:** Facilitates management by clearly documenting the decision rationale. **Learn:** Supports learning by providing a detailed explanation and rationale.

TABLE 4. Quantitative results for Case Study 1, including transparency, traceability, and MTTR improvement.

[HTML]EFEFEF **Metric**	**Result**
Transparency Score (TS)	4.6 / 5
Traceability Alignment (TA)	4.4 / 5
MTTR Reduction (AKS vs baseline)	30% improvement

c: OPERATIONAL IMPACT

The 30% % reduction in MTTR was attributed to:

- AKS auto-healing capabilities, and
- clearer decision rationale, enabling SREs to prepare appropriate monitoring and recovery configurations.

Although the improvement cannot be fully attributed to ADR-E, the participants indicated that the clarity of the rationale of the decision made the operational configuration more consistent and effective.

d: QUALITATIVE INSIGHTS

Interview themes revealed:

- strong preference for explanations addressing "What-if" scenarios;
- high perceived value in documenting rejected alternatives;
- improved alignment between technical and non-technical participants.

B. INTERPRETATION OF MTTR IMPROVEMENTS

Both case studies reported an observed reduction of approximately 30% in Mean Time to Resolution (MTTR) following the architectural decisions under study. While this improvement represents an important operational outcome, it requires careful interpretation to avoid attributing causal effects solely to the use of ADR-E.

In both cases, the main technical contributors to the reduction of MTTR were the capabilities of the selected solutions themselves. In Case Study 1, improvements were largely driven by the auto-healing, managed orchestration, and operational tooling provided by Azure Kubernetes Service. In the case Study 2, the reductions in MTTR were mainly attributable to the adoption of an enterprise observability platform, which offered unified dashboards, correlated traces and more consistent alert mechanisms.

ADR-E did not directly optimise runtime behaviour, monitoring infrastructure, or recovery mechanisms. Instead, its contribution was indirect and organisational in nature. By making the rationale for the decisions, assumptions, and the expected behaviours explicit, ADR-E supported better preparedness before incidents. Participants reported that clearer documentation of architectural intent helped them configure monitoring, alerts, and runbooks in a more consistent and aligned manner.

Furthermore, explicit documentation of trade-offs and rejected alternatives reduced ambiguity during incident analysis. When failures occurred, stakeholders were able to reason more effectively about whether observed behaviour was expected or anomalous with respect to the documented architectural decision. This reduced the time spent on misinterpretation and unnecessary exploration of discarded options.

Together, these observations suggest that ADR-E should be understood as an enabling factor rather than a direct cause of the reduction in MTTR. Its primary contribution lies in improving shared understanding, decision transparency, and cross-role alignment, which in turn support more effective operational practices. The MTTR improvements reported in this study therefore reflect the combined effect of technical capabilities and improved architectural explainability, rather than documentation changes alone.

1) THREATS TO VALIDITY

a: INTERNAL VALIDITY

Operational improvements may stem partly from increased stakeholder engagement rather than ADR-E alone. Workshops may have introduced a maturation effect.

b: EXTERNAL VALIDITY

Results may not generalize to other organizations, especially those not using Azure or lacking DevOps maturity.

c: CONSTRUCT VALIDITY

Perceived transparency and traceability are self-reported; although triangulated with artefact analysis, some bias may remain.

d: CONCLUSION VALIDITY

The sample size is small (n=6), which limits statistical robustness; however, strong qualitative convergence supports the findings.

Case Study 1 shows that applying SAEF and ADR-E improved the clarity, traceability, and stakeholder understanding of a critical architectural decision. The structured explanations and explicit rationale in ADR-E supported more informed discussions and facilitated operational preparation, partially contributing to improved MTTR. These results demonstrate the value of explainability-oriented documentation in real architectural settings.

A consolidated comparison of Case Study 1 and Case Study 2 is presented later in Section V-D, supported by Figures 4, 5, and 6.

C. CASE STUDY 2: ADOPTING AN ENTERPRISE OBSERVABILITY PLATFORM

The second case study evaluates SAEF and ADR-E in the context of selecting an enterprise-grade observability solution for a high-traffic e-commerce platform. The system comprises more than 60 microservices, handling thousands of daily transactions and subject to strict availability and performance requirements.

The core decision focused on whether to:

- adopt an **enterprise observability platform** (commercial, integrated solution), or
- continue with a **tool-based observability stack** composed of separate open-source tools.

The organization's existing observability setup relied on multiple independent tools for logs, metrics, and traces, creating fragmentation and limited end-to-end visibility. This decision had substantial implications for incident detection, diagnosis time, operational cost, and long-term maintainability. As in Case Study 1, prior decisions were documented using traditional ADRs, with limited rationale and weak support for cross-team understanding.

Applying ADR-E in this setting allowed us to assess its usefulness in a more complex environment where observability, risk, and business impact are tightly coupled.

1) ADR-E FOR THE OBSERVABILITY DECISION

Table 5 presents the ADR-E instance for the decision *Adopt an Enterprise Observability Platform*. It illustrates how the decision rationale, alternatives, consequences, and stakeholder-oriented explanations are captured in a structured and explainable way, making the decision easier to understand, review, and reuse.

2) PARTICIPANTS

Eight professionals participated in Case Study 2:

- 1 Lead Software Architect,
- 2 Senior Backend Developers,
- 2 Site Reliability Engineers (SREs),
- 1 Product Owner,
- 1 Operations Manager,
- 1 Incident Response Lead.

This composition reflects a broader stakeholder set than in Case Study 1, capturing technical, operational, and business perspectives.

3) STUDY DESIGN AND PROCEDURE

The study design mirrored the structure of Case Study 1 while adapting activities to the observability context.

a: 1) EXPLAINABILITY REQUIREMENTS ELICITATION

A workshop identified concerns related to:

- incident detection latency,
- cross-service dependency visibility,
- alert noise and false positives,
- cost vs. benefit of commercial licensing,
- operational risk during peak periods.

These concerns were mapped to ER1–ER5 and used to guide the construction of the ADR-E.

TABLE 5. ADR-E instance for the architectural decision: Adopt an enterprise observability platform (Case Study 2).

Element	Description
Title	ADR010 - Adopt an Enterprise Observability Platform
Status	Adopted
Decision	Adopt a commercial observability platform to centralize metrics, logs, and traces.
Context	The company operates a high-traffic e-commerce platform with over 60 microservices. The existing observability stack is fragmented across multiple tools, which complicates incident detection and diagnosis.
Options Considered	1. Enterprise observability platform (Adopted): Pros - Unified view, advanced analytics, vendor support. Cons - Licensing costs, vendor lock-in. 2. Open-source toolchain: Pros - Lower direct cost, high flexibility. Cons - Integration complexity, higher maintenance effort.
Consequences	Positive - Faster incident detection, improved cross-team visibility, reduced mean time to resolution (MTTR). Negative - Additional licensing costs and dependency on vendor roadmap.
Advice	Advice from SRE Lead and Head of Operations emphasized the need for unified observability and predictable support for critical incidents.
Rationale	The commercial platform was chosen due to its unified telemetry, advanced alerting, built-in dashboards, and reduced need for in-house integration work.
Version	v1.0
Date Created	2024-07-05
Date Updated	2024-07-05
Related ADRs	ADR008 - Logging Strategy ADR009 - Monitoring Policy
Explanation	The commercial observability platform provides a single-pane-of-glass for monitoring microservices, enabling faster triage of incidents and supporting business-critical SLAs.
Revision History	v1.0 (2024-07-05) - Initial creation.
Feedback and Review	Reviewed by architecture board and SRE team. Consensus reached on benefits outweighing licensing costs.
Research Questions	**Why:** Need to reduce incident resolution time and improve visibility across services. **What:** Adopt a commercial observability solution. **What-if:** If we remained with the open-source stack, integration and maintenance effort would continue to increase. **Where:** Applies to the monitoring and operations layer. **Who:** Architecture team, SREs, Operations. **When:** Decision made during the reliability improvement initiative in 2024-Q3.
Purpose for Explaining	**Justify:** Document reasons for choosing a commercial platform. **Improve:** Provide a reference for future observability-related decisions. **Evaluate:** Assess the impact on operational KPIs such as MTTR and incident rate. **Manage:** Facilitate budget discussions and vendor management by clarifying expected benefits. **Learn:** Capture lessons learned for future platform-level decisions.

b: 2) DECISION MODELLING AND ADR-E CONSTRUCTION

The architecture and SRE teams jointly:

- defined evaluation criteria (e.g., MTTR reduction potential, integration effort, support quality),
- assessed both the open-source toolchain and the enterprise platform,
- documented rationale, alternatives, and expected consequences within the ADR-E.

A traditional ADR was again created as a baseline for comparison.

c: 3) VALIDATION WORKSHOPS

Three workshops were held:

- Workshop 1: overview of current observability limitations and candidate solutions,
- Workshop 2: scenario-based evaluation (e.g., payment service failure, degraded latency),
- Workshop 3: review of ADR and ADR-E, with structured feedback from all participants.

d: 4) QUANTITATIVE ASSESSMENT

The same metrics used in Case Study 1 were applied:

- **Transparency Score (TS):** stakeholder ratings of explanation clarity (1–5).
- **Traceability Alignment (TA):** independent scoring of links between decisions, quality attributes, and related artefacts.
- **Mean Time to Resolution (MTTR):** measured using incident simulations run before and after adopting the observability platform.

e: 5) QUALITATIVE ASSESSMENT

Semi-structured interviews and workshop notes were analysed thematically to understand:

- perceived usefulness of ADR-E explanations,
- adequacy of documented trade-offs,
- confidence in the decision under stress scenarios,
- differences in how technical and business stakeholders engaged with the documentation.

4) RESULTS

Table 6 summarizes the quantitative outcomes for Case Study 2.

a: TRANSPARENCY

Participants highlighted that ADR-E made complex trade-offs—particularly around licensing cost versus operational benefit—more explicit and easier to discuss across roles.

TABLE 6. Quantitative evaluation results for Case Study 2, including transparency, traceability, and MTTR improvements.

[HTML]EFEFEF **Metric**	**Result**
Transparency Score (TS)	4.7 / 5
Traceability Alignment (TA)	4.5 / 5
MTTR Reduction (Platform vs baseline)	30% improvement

b: TRACEABILITY

Links to reliability-related quality attributes and incident management policies were considered clearer and more systematic than in prior documentation.

c: OPERATIONAL IMPACT

Similar to Case Study 1, a 30% MTTR reduction was observed. In this case, improvements were largely attributed to:

- unified dashboards and correlated traces,
- more consistent alerting rules,
- clearer decision rationale guiding SRE runbooks.

d: QUALITATIVE INSIGHTS

Interview analysis revealed:

- strong appreciation from business and operations stakeholders for the ''Purpose for Explaining'' section,
- better shared understanding of cost–benefit trade-offs,
- recognition that rejected alternatives documentation would be useful for future procurement or renegotiation cycles.

5) THREATS TO VALIDITY

a: INTERNAL VALIDITY

Some improvements may be partly due to parallel reliability initiatives running in the organization, not solely the adoption of the observability platform or ADR-E.

b: EXTERNAL VALIDITY

The findings come from a single e-commerce domain; other domains, such as safety-critical or regulated environments, may exhibit different explainability needs.

c: CONSTRUCT VALIDITY

As in Case Study 1, transparency and traceability were partly measured via self-reported scores, although these were corroborated with artefact inspection and incident metrics.

d: CONCLUSION VALIDITY

The sample size remains modest (n=8), limiting statistical power. However, the convergence between quantitative and qualitative evidence strengthens confidence in the observed effects.

Overall, Case Study 2 confirms and extends the findings from Case Study 1, showing that ADR-E and SAEF are effective in a more complex, high-stakes operational environment, and that explainability can support both technical and business decision-making in observability-related architectural choices.

D. COMPARATIVE ANALYSIS OF THE CASE STUDIES

This comparative evaluation builds upon the findings of Case Study 1 (Section V-A) and Case Study 2 (Section V-C), providing a unified interpretation of how ADR-E performs across different architectural decision contexts.

The two case studies provide complementary perspectives on the applicability and value of the Software Architecture Explainability Framework (SAEF) and the ADR-E template across distinct architectural contexts. While Case Study 1 focused on platform selection (Azure Kubernetes Service), Case Study 2 examined the adoption of an enterprise-grade observability solution in a high-traffic e-commerce environment. The comparative analysis highlights convergent benefits, contextual differences, and insights regarding the generalizability of explainability-oriented decision documentation.

To comprehensively evaluate the impact of the ADR-E framework across both case studies, we conducted a detailed comparison of outcomes based on predefined criteria aligned with the objectives of SAEF.

Table 7 provides a direct side-by-side comparison of the evaluation results from both case studies, highlighting differences in transparency, traceability, and operational impact.

Figure 4 provides an integrated overview of the outcomes observed in both case studies, synthesizing transparency, traceability, and stakeholder feedback into a single cross-case visualization. This figure introduces the comparative evaluation by highlighting the consistent improvements enabled by ADR-E across different architectural contexts.

Together with Figure 4, Figures 5 and 6 provide complementary perspectives on the cross-case results. While Figure 4 summarizes overall stakeholder perceptions, Figure 5 contrasts the relative performance of transparency, traceability, and operational impact across cases, and Figure 6 highlights finer-grained variations across evaluation criteria.

Figure 5 illustrates a radar chart comparing evaluation metrics across the two case studies. The metrics include fault detection improvement, transparency, traceability, and stakeholder satisfaction, offering a visual comparison of how ADR-E performed in diverse project contexts. Case Study 1 scored slightly higher on transparency, with a score of 4.7/5, attributed to its simpler context and narrower scope. In contrast, Case Study 2 showcased a broader application with slightly lower traceability (4.4/5), reflecting the complexity of integrating ADR-E in a high-traffic e-commerce platform.

The heatmap shown in Figure 6 complements the radar chart by providing a detailed, cell-level representation of performance across evaluation metrics. The highest-performing areas, such as transparency and stakeholder satisfaction, emphasize the efficacy of ADR-E in aligning technical decisions with organizational goals. Meanwhile, traceability,

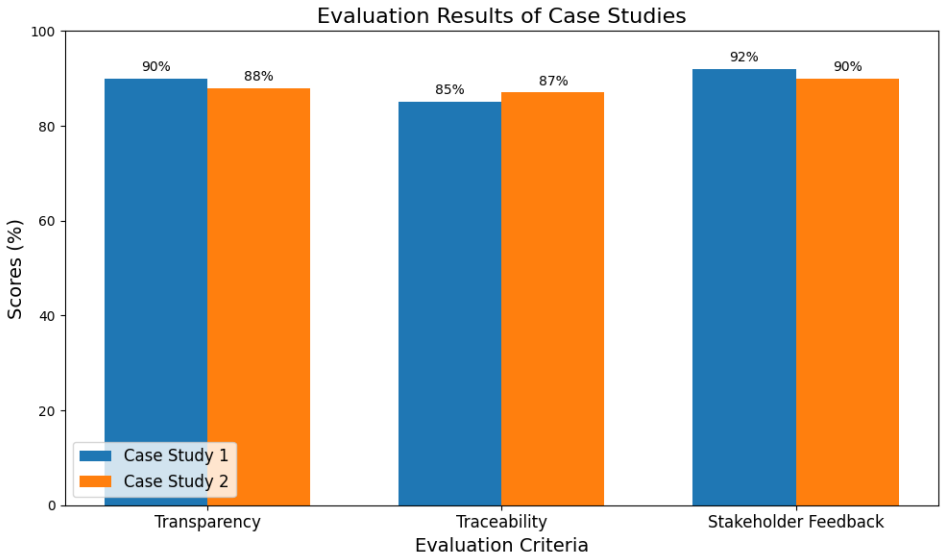

FIGURE 4. Cross-case overview of outcomes for Case Study 1 and Case Study 2, including transparency, traceability, and stakeholder feedback.

TABLE 7. Detailed comparison of transparency, traceability, and operational outcomes across both case studies.

[HTML]EFEFEF **Criterion**	**Case Study 1**	**Case Study 2**	**Outcome**
Transparency			
Stakeholder understanding	4.8/5	4.6/5	High trust
Completeness of ADR-E	4.6/5	4.5/5	Detailed documentation
Traceability			
Linkage to artifacts	4.6/5	4.5/5	Better decision dependencies
Ease of navigation	4.4/5	4.3/5	Positive feedback on decision traceability
Stakeholder Feedback			
Qualitative sentiment	Positive	Positive	Alignment with expectations
Engagement level	High	Moderate	Case 1 had more proactive involvement

TABLE 8. Overall impact of ADR-E implementation across both case studies, summarizing improvements in documentation quality, stakeholder alignment, and operational performance.

Table 8 synthesizes the global impact of ADR-E across both studies, complementing the comparative results from Table 7.

[HTML]EFEFEF **Metric**	**Case Study 1 Outcome**	**Case Study 2 Outcome**
Mean Time to Resolution (MTTR)	30% reduction after implementation.	30% reduction under high-traffic conditions.
Stakeholder Satisfaction	4.7/5 average survey score.	4.6/5 average survey score.
Documentation Depth	Comprehensive ADR-E templates led to fewer dependencies.	Iterative documentation refinements improved clarity.
Scalability Improvement	Successfully handled a 20% traffic increase.	Managed a 25% traffic increase seamlessly.

while strong, highlights the need for more streamlined linkage mechanisms between decisions and related artifacts. Together, these figures underscore the framework's strengths and areas for refinement, illustrating its adaptability to varying project complexities.

1) CONVERGENT FINDINGS ACROSS BOTH CASES

a: 1) IMPROVED TRANSPARENCY AND STAKEHOLDER COMPREHENSION

Across both case studies, stakeholders rated ADR-E significantly higher in explanation clarity compared to traditional ADRs. Transparency scores were similar (4.6 and 4.7), suggesting that the structured explanation elements of ADR-E—particularly the "Why" and "What-if" questions—consistently supported understanding for both technical and non-technical participants.

Participants in both organizations reported that ADR-E offered:

- clearer justification of rationale,
- better articulation of rejected alternatives,
- improved comprehension of trade-offs,
- and more accessible explanations for operational and business stakeholders.

This convergence indicates that explainability requirements (ER1–ER5) are broadly shared across different types of architectural decisions.

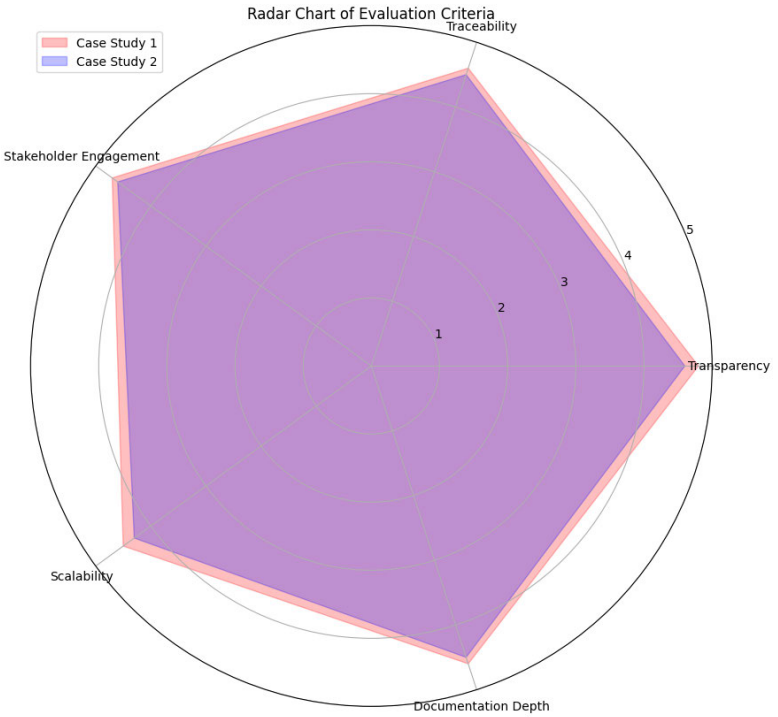

FIGURE 5. Radar chart comparing key evaluation criteria across case studies.

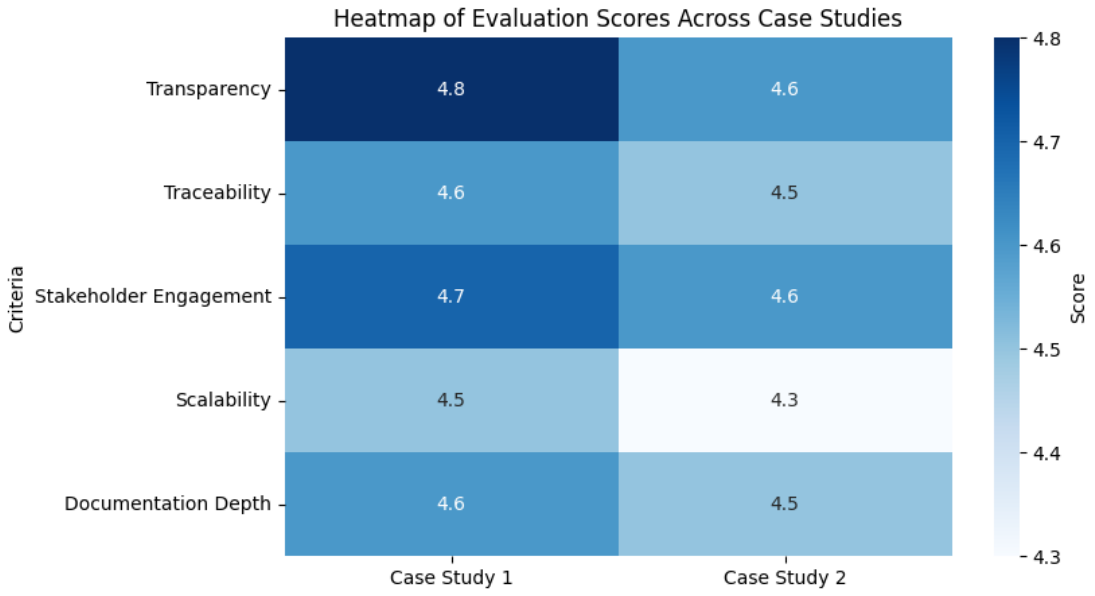

FIGURE 6. Heatmap showing evaluation scores across criteria for both case studies.

b: 2) STRENGTHENED TRACEABILITY AND DECISION COHERENCE

Both studies demonstrated higher traceability completeness when using ADR-E. Traceability Alignment (TA) scores were highly consistent (4.4–4.5), showing that ADR-E's explicit linking of decisions to concerns, quality attributes, and related ADRs is valuable regardless of decision domain.

The metamodel elements (e.g., links to QAs, dependencies, and impacted components) played a central role in improving decision coherence in both cases.

c: 3) FACILITATION OF STAKEHOLDER ALIGNMENT

In both organizations, ADR-E facilitated cross-team communication. SREs, platform engineers, and business representatives highlighted that structured explanations reduced ambiguity and improved mutual understanding.

This confirms SAEF's aim of providing stakeholder-oriented explanations that mitigate the disconnects typically observed in traditional decision documentation.

2) CONTEXTUAL DIFFERENCES AND DECISION-SPECIFIC INSIGHTS

Despite the convergent benefits, some differences emerged due to the nature of each decision and organizational context.

a: 1) OPERATIONAL IMPACT

Both case studies reported a **30% MTTR reduction**, but the mechanisms differed:

- In **Case Study 1**, the improvement was driven mainly by AKS's auto-healing features combined with clearer configuration guidance derived from ADR-E.
- In **Case Study 2**, enhanced observability tooling and more effective alert correlation were the primary contributors.

In both scenarios, participants noted that ADR-E improved readiness by clarifying assumptions and expected behaviours, but the operational gains were not solely attributable to documentation improvements.

b: 2) DECISION COMPLEXITY AND INFORMATION LOAD

Case Study 2 involved significantly higher system scale (60+ microservices). Participants reported that ADR-E's structured explanation helped prevent cognitive overload by organizing information across multiple layers (logs, metrics, traces, costs, and scalability factors).

In Case Study 1, the decision scope was narrower, and ADR-E served mainly to clarify trade-offs and stakeholder priorities.

This suggests ADR-E is particularly beneficial as decision complexity increases.

c: 3) BUSINESS VS. TECHNICAL STAKEHOLDER NEEDS

Case Study 2 revealed a greater emphasis on business-related concerns (e.g., licensing costs, operational risk during peak traffic). The "Purpose for Explaining" section in ADR-E was frequently referenced by non-technical stakeholders.

In contrast, Case Study 1 had a stronger technical focus, with stakeholders valuing clarity around deployment models, scaling policies, and operational implications.

Thus, ADR-E demonstrated flexibility to address heterogeneous stakeholder needs.

3) CROSS-CASE LESSONS FOR SAEF AND ADR-E

Several overarching lessons emerged:

- **Structured explanations are consistently valuable.** Regardless of domain, stakeholders emphasized the importance of the Why/What-if reasoning prompts.
- **Traceability is an area of clear improvement.** ADR-E made dependencies, impacts, and quality attributes more explicit in both settings.
- **Decision complexity increases the relative benefit of ADR-E.** The more multidimensional the decision, the more stakeholders appreciated the structured rationale.
- **Explainability enhances preparedness for operational scenarios.** Stakeholders in both cases tied better explanations to improved readiness and configuration quality.
- **Documentation alone does not guarantee performance improvements.** While operational metrics improved, ADR-E contributed indirectly through clarity and configuration—not through technical optimizations.

The comparative analysis shows that SAEF and ADR-E provide consistent benefits across distinct architectural decision contexts. Both case studies confirmed improvements in transparency, traceability, and stakeholder alignment, while also revealing contextual nuances in how explainability contributes to operational readiness and comprehension. These findings strengthen the argument that integrating explainability into architectural decisions can enhance decision quality and knowledge continuity across diverse software systems.

VI. DISCUSSION

This section discusses the broader meaning of the results obtained from applying the Software Architecture Explainability Framework (SAEF) and the ADR-E template in two industrial case studies. Our discussion synthesises the evidence, interprets the outcomes, and highlights lessons learned, limitations, and implications for both research and practice.

A. INTERPRETATION OF FINDINGS

Across both case studies, SAEF and ADR-E demonstrated consistent benefits in transparency, traceability, and stakeholder comprehension. The high transparency scores (4.6–4.7) reflect the usefulness of structured explanations—particularly the "Why" and "What-if" prompts—for improving understanding of architectural decisions. Stakeholders repeatedly emphasised that ADR-E made the rationale and trade-offs more accessible, reducing ambiguity and improving cross-team communication.

Traceability improvements were similarly consistent, supported by the explicit linking of decisions to related ADRs, quality attributes, and stakeholder concerns. In previous documentation practices, such relationships were often implicit or scattered across artefacts. ADR-E facilitated a more coherent and navigable decision landscape, which was particularly valuable in the more complex environment of Case Study 2.

The operational outcomes, notably the 30% reduction in MTTR reported in both cases, require nuanced interpretation. While these improvements cannot be solely attributed to ADR-E, participants consistently linked parts of these gains to clearer decision rationale, better alignment between teams, and more informed configuration choices. Thus, the evidence suggests that explainability contributes indirectly to operational performance by enhancing preparedness and reducing misinterpretations.

B. CROSS-CASE LESSONS LEARNED

The comparison between the two case studies highlights several overarching lessons:

1) EXPLAINABILITY SCALES WITH DECISION COMPLEXITY

The more complex the decision environment, the more stakeholders appreciated the structure provided by ADR-E. In Case Study 2, the number of components, quality attributes, and operational dependencies made structured explanations indispensable.

2) STAKEHOLDER DIVERSITY INCREASES THE NEED FOR EXPLAINABILITY

ADR-E proved effective in accommodating the needs of both technical and non-technical participants. Business and operations stakeholders found the "Purpose for Explaining" and scenario-based reasoning particularly helpful.

3) REJECTED ALTERNATIVES ARE CRITICAL FOR TRANSPARENCY

A recurring theme in both studies was the importance of documenting why alternatives were rejected. This element, often missing in traditional ADRs, significantly improved stakeholder alignment and reduced repeated discussions.

4) EXPLAINABILITY ENHANCES ARCHITECTURAL LEARNING

Participants viewed ADR-Es as reusable knowledge artefacts. Teams revisited explanations to support new decisions, illustrating how explainability contributes to long-term organizational learning and architectural continuity.

C. CHALLENGES AND AREAS FOR IMPROVEMENT

Despite its benefits, SAEF also revealed practical challenges:

1) DOCUMENTATION OVERHEAD

Participants noted that ADR-E requires more effort than traditional ADRs. Without tooling support, this overhead may discourage adoption, especially in fast-paced environments.

2) NEED FOR STANDARDIZATION AND GUIDANCE

Differences in how teams authored ADR-E entries showed that clearer guidelines, examples, and usage patterns are needed to ensure consistent application across projects.

3) INTEGRATION WITH DEVELOPMENT TOOLCHAINS

Teams highlighted the importance of integrating ADR-E with existing workflows (e.g., Git, Jira, Confluence). Without seamless integration, explainability activities risk becoming siloed or inconsistently maintained.

4) MAINTAINING CURRENCY OF DECISIONS

Explainability is only useful if explanations remain accurate. Some participants emphasized the need for mechanisms to periodically review and update ADR-E entries, especially as systems evolve.

D. BROADER IMPLICATIONS FOR PRACTICE

The findings offer several implications for software architecture practice:

- **Improved communication and alignment:** SAEF supports clearer reasoning about architectural choices, reducing misunderstandings and increasing trust among stakeholders.
- **Better preparedness and operational readiness:** Structured explanations help teams anticipate failure scenarios and configure systems appropriately.
- **Long-term architectural sustainability:** ADR-E contributes to a reusable knowledge base that supports onboarding, audits, and informed evolution of systems.
- **Support for decision governance:** Explainability aligns well with governance processes in regulated industries requiring auditability and accountability.

E. IMPLICATIONS FOR RESEARCH

From a research perspective, the results reveal promising directions:

- **AI-assisted explainability in architecture:** Future work may explore automated generation of explanations or decision alternatives using LLMs or AI-driven tools.
- **Tool support for ADR-E:** Dedicated tools could reduce documentation overhead, enforce consistency, and automate traceability links.
- **Longitudinal impact studies:** Understanding how explainability influences architectural evolution, technical debt, and team behaviour over time requires longer-term studies.
- **Cross-domain applications:** Applying SAEF to safety-critical, data-intensive, or cyber-physical systems could reveal domain-specific explainability requirements.

In general, the two case studies demonstrate that SAEF and the ADR-E template offer practical value in achieving greater transparency, traceability, and stakeholder alignment in architectural decision-making. At the same time, the identified challenges identified—documentation effort, the need for standardisation, and integration into existing workflows—indicate important directions for improvement. Addressing these issues will be essential to fully realise the potential of explainable architectural decision-making in modern software systems.

F. COMPARISON OF ADR-E WITH EXISTING ADR APPROACHES

This subsection discusses how the ADR-E template is related to and extends existing ADR approaches, using the same explainability-orientated criteria introduced in Section II (Table 1). The purpose of this comparison is to clarify how ADR-E operationalises explainability at the level of architectural decision documentation, addressing limitations identified in established practices.

1) COMPARISON USING EXPLAINABILITY-ORIENTED CRITERIA

Core decision metadata and decision statement (D1–D2). ADR-E retains all core elements present in traditional ADR templates, including explicit decision identifiers, status, versioning information, and clear decision statements. This ensures compatibility with existing ADR practices and facilitates incremental adoption without disrupting established documentation processes.

Context and alternatives (D3–D4). As with ADR and MADR, ADR-E provides explicit sections to describe the decision context and enumerate alternative solutions. ADR-E extends these elements by requiring architects to document the assumptions, constraints, and decision drivers that underlie the evaluation of alternatives, thus improving the interpretability of the documented context.

Rejected alternatives and justification (D5). Unlike the ADR approaches reviewed in Section II, ADR-E explicitly requires the documentation of the rejected alternatives together with the rationale for their exclusion. This supports transparency in architectural reasoning and reduces the risk of knowledge loss or repeated re-evaluation of previously discarded options during system evolution.

Consequences and trade-offs (D6). ADR-E preserves the explicit documentation of consequences found in traditional ADRs, while encouraging a more systematic articulation of both positive and negative impacts. By explicitly linking consequences with quality attributes and risks, ADR-E supports clearer reasoning about architectural trade-offs and longer-term implications.

Traceability to related decisions and artefacts (D7). A distinguishing feature of ADR-E is its explicit support for traceability. ADRs-E introduces dedicated fields to link decisions to related ADRs, requirements, risks, and architectural artefacts. This enables navigation across architectural knowledge and supports impact analysis, which is not structurally supported by the ADR approaches reviewed in Section II.

Stakeholder-oriented explanation prompts (D8). ADR-E systematically incorporates stakeholder-oriented explanation prompts, such as *Why*, *What-if*, *Who*, *Where*, and *When*. These prompts are intended to make architectural decisions understandable to heterogeneous stakeholders, rather than assuming a single technical audience.

Purpose for explaining a decision (D9). ADR-E explicitly captures the purpose of explaining a decision, distinguishing between intents such as justification, evaluation, management, and learning. This capability is absent from existing ADR approaches and supports clearer communication, review, and reuse of architectural decisions over time.

2) IMPLICATIONS OF THE COMPARISON

Taken together, this comparison shows that ADR-E does not replace existing ADR approaches, but extends them by operationalising explainability requirements that are not structurally supported in current ADR templates. Although traditional ADR are effective in recording architectural decisions, ADR-E broadens their scope by making rationale, stakeholder concerns, and decision interdependencies explicit and reviewable. In doing so, ADR-E complements prior work on architectural decision documentation and provides a concrete mechanism for embedding explainability into architectural practice.

VII. THREATS TO VALIDITY

As with any empirical investigation in software engineering, this study is subject to several threats to validity. We address them following the well-established categories of internal, external, construct, and conclusion validity.

A. INTERNAL VALIDITY

Internal validity concerns the degree to which the observed outcomes can be attributed to the application of SAEF and ADR-E rather than external or uncontrolled factors. Although both case studies followed a structured procedure, several factors may have influenced results:

- **Maturation and Learning Effects:** Workshops and collaborative sessions may have improved team coordination independently of ADR-E. Increased familiarity with the problem domain could also influence the results.
- **Tooling Differences:** Operational improvements (e.g., MTTR reduction) partly derive from the systems selected (e.g., AKS or enterprise observability tools). ADR-E likely contributed indirectly by improving clarity and configuration, but not exclusively.
- **Researcher Involvement:** The authors facilitated some activities, which may have increased stakeholder engagement or introduced subtle bias during elicitation or explanation phases.

Mitigation strategies included triangulation across artefacts, interviews, and surveys, and separating facilitation roles from evaluation where possible.

B. EXTERNAL VALIDITY

External validity concerns the generalizability of the findings beyond the studied settings. Both case studies were conducted in organizations with established DevOps practices and relatively mature architectural processes. Threats include:

- **Organizational Context:** Companies with less mature documentation cultures or without continuous deployment pipelines may experience different outcomes.
- **Technology Stack Specificity:** Decisions involved Azure and Kubernetes in Case Study 1, and an enterprise observability platform in Case Study 2. Other stacks (AWS, GCP, on-premise systems) may shape explainability needs differently.
- **Domain-Specific Concerns:** Results in e-commerce or cloud-native contexts may not fully extend to safety-critical, defence, or embedded systems, where explainability requirements and constraints differ.

Nevertheless, the core explainability principles in SAEF (structured rationale, stakeholder-oriented explanations, traceability) are domain-agnostic, supporting cautious generalization.

C. CONSTRUCT VALIDITY

Construct validity assesses whether the study accurately measured the constructs of interest—transparency, traceability, and stakeholder comprehension.

- **Self-Reported Measures:** Transparency and comprehension scores were surveyed using Likert scales, which may be subject to social desirability bias or respondent interpretation differences.
- **Traceability Scoring Subjectivity:** Scoring completeness of traceability links involved human judgment, although independent reviewers were used to mitigate bias.
- **Operational Metrics Attribution:** MTTR improvements serve as partial proxies for architectural clarity and readiness, but naturally depend on other operational factors (infrastructure, tooling, traffic).

Construct threats were mitigated through triangulation: survey data, interview themes, artefact inspection, and operational logs.

D. CONCLUSION VALIDITY

Conclusion validity relates to the soundness of inferences drawn from the data.

- **Sample Size:** Both studies involved a modest number of participants, limiting statistical inference.
- **Non-Random Assignment:** Participants were selected from active project teams, which could introduce selection bias.
- **Correlation vs. Causation:** Improvements in comprehension, traceability, or MTTR cannot be attributed solely to ADR-E; the methodology supports but does not guarantee causality.

Despite these limits, convergence of qualitative and quantitative evidence across two independent cases strengthens confidence in the findings' reliability.

In summary, while the study is subject to typical limitations of empirical research in real-world software engineering contexts, methodological triangulation, multi-role participation, and artefact-based evaluation help to strengthen the validity of the results. Future replication studies in different domains and with larger sample sizes will further improve external and conclusion validity.

VIII. FUTURE WORK

This work represents the first step toward establishing a foundation for explainable architectural decision-making. While SAEF and the ADR-E template provide a structured approach for documenting and communicating decisions, significant opportunities remain to extend these contributions into more advanced decision analysis, visualization, and automation capabilities. A unifying principle of all future directions is that **the ADR-E serves as the fundamental unit of architectural knowledge**: every advanced tool, model, or analytical pipeline must start from this structured artefact.

A. 1. DECISION GRAPHS CONSTRUCTED FROM ADR-E ARTEFACTS

A key research direction is the construction of a **Decision Graph**, where ADR-Es serve as nodes and their relationships (dependencies, conflicts, refinements, and impacts) form the edges. Such a graph would enable architects to:

- visualize the evolution and interconnectedness of decisions across the lifecycle,
- perform impact analysis by traversing related decisions,
- detect decision smells (e.g., orphan decisions or cyclical reasoning),
- identify clusters of decisions affecting specific quality attributes or components.

Because ADR-Es include explicit rationale, alternatives, and traceability links, they are ideal as the *unitary building blocks* for creating reliable decision graphs. This graph-based representation lays the groundwork for automated reasoning and large-scale architectural knowledge analysis.

B. 2. LLM-BASED AUTOMATION AND DECISION ANALYSIS

Once decisions are represented in graph form, modern **Large Language Models (LLMs)** can be leveraged to assist architects in advanced tasks, such as:

- summarizing decision clusters or evolution paths,
- detecting inconsistencies or missing rationale across ADR-Es,
- suggesting alternative solutions based on prior decisions,
- generating candidate ADR-E drafts from architectural discussions or design documents,
- performing automated traceability recovery between requirements, risks, and decisions.

LLMs are particularly well-suited to reasoning over structured natural language artefacts such as ADR-Es. However, this is only possible if decisions are captured in a systematic way—reinforcing the importance of ADR-E as the atomic element for analysis.

C. 3. TOOL SUPPORT FOR MANAGING EXPLAINABLE ADRS

To facilitate adoption in real-world projects, a dedicated tool for managing ADR-Es is needed. This tool should:

- provide integrated version control, enabling a complete evolution history,
- maintain traceability links across requirements, design artefacts, and code,
- support collaborative authoring and review processes,
- allow automatic generation and analysis of decision graphs,
- offer LLM-based assistance for explanation refinement and decision exploration.

Such tooling would operationalize SAEF and make explainable decision-making practical and sustainable in large-scale or fast-paced environments.

D. 4. GRAPHICAL NOTATION FOR EXPLAINABLE DECISION-MAKING

A complementary research direction is the development of a **graphical notation** that visually represents:

- the structure of an ADR-E,
- relationships between decisions,
- decision impacts and quality attribute trade-offs,
- temporal evolution and versioning.

This visual language would reduce cognitive effort for stakeholders and help communicate architectural reasoning more effectively, particularly in cross-functional teams.

E. 5. IMPACT-BASED DECISION GUIDELINES

Finally, future work will extend SAEF with **impact-based decision guidelines**. These guidelines will include:

- a structured framework for assessing technical and organizational consequences,
- criteria for evaluating alternatives using impact scenarios,
- documentation patterns for recording consequences in ADR-Es,
- processes for continuously re-evaluating decisions under evolving requirements.

Integrating systematic impact analysis into ADR-E entries will strengthen the robustness and defensibility of architectural decisions.

Overall, future work will expand SAEF from a documentation and communication framework into a **full decision intelligence ecosystem**. Central to this vision is the role of ADR-E as the *atomic unit* of all higher-level reasoning—enabling graph-based analysis, LLM-assisted automation, advanced visualization, and evidence-based decision support. These developments will elevate explainable architecture from a documentation practice to an analytical capability, strengthening transparency, accountability, and resilience in modern software systems.

IX. CONCLUSION

This work introduced the **Software Architecture Explainability Framework (SAEF)**, a structured approach for improving the transparency, traceability, and comprehensibility of architectural decisions. At the core of the framework is the **Enhanced Architectural Decision Record (ADR-E)**, which extends traditional ADR templates with explicit rationale, structured explanations, rejected alternatives, traceability links, and stakeholder-oriented reasoning. Drawing inspiration from principles in explainable AI, SAEF provides a systematic means to articulate and communicate architectural decisions in a way that addresses the needs of diverse stakeholders.

The empirical evaluation through two industrial case studies demonstrated the practical value of SAEF. ADR-E improved stakeholders' understanding of architectural trade-offs, strengthened traceability across related decisions, and contributed to better operational preparedness. Although performance improvements such as reduced MTTR stemmed partly from the technical solutions adopted, the clarity provided by ADR-E played an important supporting role in enabling informed configuration and cross-team alignment. Taken together, the findings indicate that explainability enhances not only documentation quality but also broader architectural practices.

The study also revealed directions for further refinement. Effective adoption of ADR-E requires minimizing documentation overhead, improving consistency, and integrating explainability practices into existing development workflows. Future work will therefore focus on:

- developing tool support for ADR-E creation, versioning, and automated traceability;
- designing graphical notations that complement textual explanations and improve accessibility;
- conducting longitudinal studies to assess the long-term impact of explainable decision-making on system evolution and technical debt.

In summary, SAEF contributes a novel and actionable perspective on architectural decision-making, addressing long-standing challenges related to undocumented rationale, fragmented knowledge, and limited stakeholder alignment. By embedding explainability directly into architectural processes, the framework supports the development of software systems that are more maintainable, transparent, and resilient. This work establishes a foundation for future research on explainable software architecture and offers practical guidance for organizations seeking to strengthen their decision-making practices.

APPENDIX A
PRACTICAL GUIDELINES FOR PRACTITIONERS

To support the adoption of the Software Architecture Explainability Framework (SAEF) and the ADR-E template in real software projects, this section provides a set of practical

guidelines distilled from the two industrial case studies and the empirical evaluation. These guidelines are intended for software architects, technical leads, platform engineers, and organizations seeking to improve the transparency, traceability, and communicability of architectural decisions.

A. 1. START BY CAPTURING EXPLAINABILITY REQUIREMENTS

Before documenting a decision, identify the questions stakeholders are likely to ask:

- Why was this decision made?
- What alternatives were evaluated and why were they rejected?
- Who is affected and how?
- What-if this decision changes in the future?

Mapping these questions to explainability requirements (ER1–ER5) helps architects anticipate concerns early and produce explanations that resonate with diverse audiences.

B. 2. USE ADR-E FOR DECISIONS WITH LONG-TERM IMPACT

Not every decision requires full explainability documentation. Prioritize ADR-E for decisions that:

- affect core architectural structures,
- involve significant trade-offs or risks,
- have long-term operational or financial consequences,
- require cross-team alignment or executive approval.

This selective approach maximizes benefits while minimizing documentation overhead.

C. 3. DOCUMENT REJECTED ALTERNATIVES AND THEIR RATIONALE

Both case studies showed that documenting rejected alternatives is critical for preventing repeated discussions and avoiding re-evaluation of discarded options. Include:

- why an alternative was rejected,
- under which assumptions the alternative might become viable again,
- its potential risks or limitations.

This practice improves institutional memory and strengthens decision accountability.

D. 4. LINK DECISIONS TO QUALITY ATTRIBUTES AND RELATED ADRS

Traceability is essential for understanding systemic impacts. Ensure each ADR-E includes:

- explicit quality attributes influenced by the decision,
- dependencies or conflicts with other ADRs,
- references to requirements, risks, and architectural drivers.

Tools such as Git, Confluence, or ADR Managers can be used to maintain these links across artifacts.

E. 5. TAILOR EXPLANATIONS TO STAKEHOLDER PROFILES

Use different levels of abstraction for:

- **Technical stakeholders:** details on models, trade-offs, constraints, operational impacts.
- **Business stakeholders:** cost reasons, alignment with strategic goals, risk implications.
- **Operations/SRE teams:** failure modes, monitoring requirements, what-if scenarios.

Structured prompts in ADR-E (Why, What-if, Who, When) naturally support this tailoring.

F. 6. INTEGRATE EXPLAINABILITY INTO EXISTING WORKFLOWS

Explainability is most effective when embedded into routine processes:

- incorporate ADR-E discussions into design reviews,
- store ADR-E entries alongside code repositories,
- use pull requests or architectural decision reviews involving ADR-E updates,
- automate versioning through Git.

Seamless integration reduces resistance and ensures sustainable usage.

G. 7. PERIODICALLY REVISIT AND UPDATE ADR-E ENTRIES

Architectural decisions evolve as systems mature. Schedule periodic reviews to:

- confirm assumptions still hold,
- update implications based on new requirements,
- record consequences observed during operation,
- retire decisions that are no longer relevant.

This maintains the accuracy, usefulness, and trustworthiness of the decision repository.

H. 8. USE SAEF TO SUPPORT OPERATIONAL PREPAREDNESS

The case studies showed that explainability improves teams' readiness for incidents. When documenting decisions:

- explicitly state expected failure modes,
- describe how the decision supports mitigation or recovery,
- link ADR-E to observability dashboards or runbooks,
- document what-if scenarios in measurable terms.

Explainability becomes an enabler for resilience and operational excellence.

I. SUMMARY

These guidelines aim to help practitioners adopt explainability-oriented architectural practices effectively. By focusing on stakeholder needs, documenting rationale rigorously, and integrating explainability into daily workflows, organizations can build more transparent, robust, and maintainable software architectures that support long-term evolution and strategic alignment.

APPENDIX B
EMPIRICAL INSTRUMENTS USED IN THE EVALUATION

This appendix documents the empirical instruments used in the mixed-methods evaluation of the Software Architecture Explainability Framework (SAEF) and the ADR-E template. The instruments are provided to support transparency, reproducibility, and reuse in future studies.

A. SURVEY QUESTIONNAIRE

A post-activity survey was administered to participants after they completed the ADR-E documentation and validation workshops. All elements were measured using a 5-point Likert scale (1 = Strongly disagree, 5 = Strongly agree).

1) TRANSPARENCY AND COMPREHENSION

- Q1: The architectural decision was easy to understand.
- Q2: The rationale behind the decision was clearly explained.
- Q3: The documented explanation helped me understand why this option was selected.
- Q4: Decision documentation reduced ambiguity compared to prior documentation practices.

2) TRACEABILITY AND STRUCTURE

- Q5: Decision documentation clearly links the decision to relevant quality attributes.
- Q6: The relationships between this decision and other architectural decisions were easy to identify.
- Q7: The structure of the decision record supported navigation and review.

3) STAKEHOLDER ALIGNMENT

- Q8: The explanations were appropriate for my role and responsibilities.
- Q9: The documentation supported constructive discussions among different stakeholders.

4) OVERALL PERCEPTION

- Q10: The explainability-oriented documentation improved my confidence in the architectural decision.
- Q11: I would recommend using this form of decision documentation in future projects.

B. SEMI-STRUCTURED INTERVIEW GUIDE

Semi-structured interviews were conducted after the survey to obtain deeper qualitative insights into the' perceptions of the participants. The interviews lasted approximately 20–30 minutes and followed the guide below.

- I1: How did the decision documentation influence your understanding of the architectural choice?
- I2: Which parts of the decision record were most useful to you and why?
- I3: Were there any aspects of the explanation that were unclear or unnecessary?
- I4: How did the documentation support (or hinder) discussion among different roles?
- I5: In what ways did the documentation differ from the decision records you have used previously?
- I6: Do you think documenting rejected alternatives was useful? Why or why not?
- I7: How could the decision documentation be improved for future use?

Follow-up questions were used when necessary to clarify responses or explore emerging themes.

C. WORKSHOP AGENDA AND PHASES

Structured workshops were used to elicit explainability requirements, validate ADR-E entries, and collect feedback. Each case study involved three workshops, organised as follows.

1) WORKSHOP 1: EXPLAINABILITY REQUIREMENTS ELICITATION

- Identification of stakeholders and roles.
- Elicitation of concerns, expectations, and decision drivers.
- Map stakeholder questions to explainability requirements (ER1–ER5).

2) WORKSHOP 2: DECISION MODELLING AND SCENARIO EVALUATION

- Presentation of candidate architectural alternatives.
- Discussion of trade-offs and assumptions.
- Scenario-based evaluation (e.g., failure, scaling, operational stress).
- Construction of the ADR-E entry.

3) WORKSHOP 3: VALIDATION AND FEEDBACK

- Review of ADR-E documentation.
- Comparison with prior decision documentation practices.
- Group discussion on clarity, traceability, and usefulness.
- Collect of feedback for refinement.

These workshops ensured that both technical and non-technical perspectives were represented throughout the decision-making and evaluation process.

REFERENCES

[1] A. Jansen and J. Bosch, "Software architecture as a set of architectural design decisions," *Software*, vol. 23, no. 4, pp. 109–120, 2006.
[2] D. Falessi, G. Cantone, R. Kazman, and P. Kruchten, "Decision-making techniques for software architecture design: A comparative survey," *ACM Comput. Surv.*, vol. 43, p. 33, Oct. 2011.
[3] A. B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila, and F. Herrera, "Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Inf. Fusion*, vol. 58, pp. 82–115, Jun. 2020.
[4] R. Dwivedi, D. Dave, H. Naik, S. Singhal, R. Omer, P. Patel, B. Qian, Z. Wen, T. Shah, G. Morgan, and R. Ranjan, "Explainable AI (XAI): Core ideas, techniques, and solutions," *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–33, Jan. 2023.

[5] F. Alongi, M. M. Bersani, N. Ghielmetti, R. Mirandola, and D. A. Tamburri, "Event-sourced, observable software architectures: An experience report," *Softw., Pract. Exper.*, vol. 52, no. 10, pp. 2127–2151, Oct. 2022.
[6] J. Tyree and A. Akerman, "Architectural decisions: Why, when, and how," *IEEE Softw.*, vol. 22, no. 2, pp. 19–27, Mar. 2005.
[7] O. Zimmermann, N. Schuster, and T. Kühne, "Architectural decision guidance across projects," in *Proc. Eur. Conf. Pattern Lang. Programs (EuroPLoP)*, 2012, pp. 111–139.
[8] M. M. Morshed, M. Hasan, and M. Rokonuzzaman, "Software architecture decision-making practices and recommendations," in *Advances in Computer Communication and Computational Sciences*, S. K. Bhatia, S. Tiwari, K. K. Mishra, and M. C. Trivedi, Eds., Singapore: Springer, 2019, pp. 3–9.
[9] V. Rekha and H. c. Muccini, "Suitability of software architecture decision making methods for group decisions," in *Software Architecture*, P. Avgeriou and U. Zdun, Eds., Cham, Switzerland: Springer, 2014, pp. 17–32.
[10] A. Jansen, J. Bosch, and J. van der Ven, "Viewpoints, perspectives, and stakeholder interests in software architecture," *J. Syst. Softw.*, vol. 80, no. 9, pp. 1548–1567, 2007.
[11] A. Kostovska, D. Vermetten, S. Džeroski, P. Panov, T. Eftimov, and C. Doerr, "Using knowledge graphs for performance prediction of modular optimization algorithms," 2023, *arXiv:2301.09876*.
[12] L. H. Gilpin, D. Bau, B. Z. Yuan, A. Bajwa, M. Specter, and L. Kagal, "Explaining explanations: An overview of interpretability of machine learning," in *Proc. IEEE 5th Int. Conf. Data Sci. Adv. Anal. (DSAA)*, Oct. 2018, pp. 80–89.
[13] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 1135–1144.
[14] S. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4768–4777.
[15] O. Kopp, A. Armbruster, and O. Zimmermann, "Markdown architectural decision records: Format and tool support," in *Proc. ZEUS*, 2018, pp. 55–62.

RICARDO GACITÚA received the B.Eng. degree in informatics from Universidad de Concepción, Chile, in 1996, the M.S. degree in industrial engineering from Universidad de Chile, in 2004, and the Ph.D. degree in computer science from Lancaster University, U.K., in 2009. He is currently a Professor of informatics with Universidad de La Frontera, Chile. His research interests include software engineering, requirements engineering, ontology learning, text mining, and artificial intelligence. He was a fellow of the British Computer Society (FBCS), in 2010. He has chaired the 36th International Conference of the Chilean Computer Science Society (SCCC 2017) and the Latin-American Symposium on Software Engineering (CIbSE 2016). His awards and honors include the ORSA Award (Overseas Research Students Awards Scheme), U.K., and the British Computer Society Specialist Group on Artificial Intelligence-Outstanding Paper.

JAVIER PEREIRA received the bachelor's degree in computer science (ingeniería civil en informática) from Universidad Técnica Federico Santa María, Chile, in 1990, and the master's degree in DEA méthodes scientifiques de gestion and the Ph.D. degree in sciences de gestion from Université Paris-Dauphine (Paris IX), France, in 1992 and 1995, respectively.

He has extensive academic and industrial experience in multi-criteria and behavioral decision analysis. His work spans the modeling and development of decision-support systems, multi-criteria decision analysis for supply chain management, recommender systems, and advanced data analytics. Throughout his career, he has contributed to projects across diverse sectors, including the automotive industry, banking, healthcare, medical services, entertainment, insurance, water and sanitation services, mining, and cultural heritage. His interdisciplinary expertise and applied research trajectory reflect a strong commitment to bridging decision science with real-world organizational challenges.

MICHAEL KLAFFT received the Diploma degree in industrial engineering from Darmstadt Technical University, in 2002, and the Ph.D. degree in information systems from Humboldt University, Berlin, in 2008. He is currently a Professor of information systems and the Director of the Information Systems Institute, Jade University of Applied Sciences, Wilhelmshaven, Germany. He is also a Senior Scientist with the Fraunhofer Institute for Open Communication Systems (FOKUS), Berlin, Germany. Another field of his research is risk and crisis communication with citizens via digital media. He worked in applied research projects with enterprises from the transportation sector, the insurance sector, and meteorological service providers. One key aspect of his projects was to enhance the resilience of citizens, society, and the economy in case of disasters and extreme weather events, in cooperation with academia and relevant authorities. His research interests include requirements engineering and acceptance studies related to the use of IT systems in disaster management.